

POLITECHNIKA
GDAŃSKA

Narzędzia i Aplikacje Jakarta EE

Jakarta Persistence

Michał Wójcik

Specyfikacja Jakarta Persistence

Jakarta Persistence (wcześniej Java Persistence API - JPA):

- specyfikacja dla bibliotek odwzorowania obiektowo-relacyjnego,
- wykorzystywana w wielu frameworkach i środowiskach:
  - Java SE, Java EE, Spring Framework, Play Framework;
- nie wymaga budowania skomplikowanych obiektów DAO (ang. Data Access Object),
- obsługuje transakcje ACID (Atomicity, Consistency, Isolation, Durability),
- niezależna od dostawcy bazy danych:
  - sterowniki JDBC dla wszystkich popularnych baz,
  - przy prostszych przypadkach możliwość uniknięcia zabawy z SQL.

JPA - implementacje

JPA jest jedynie **standardem**, istnieje kilka **implementacji**:

- Hibernate, od Red Hat (używana w serwerze WildFly),
- Toplink od Oracle,
- OpenJPA od Apache Software Foundation,
- EclipseLink od Eclipse Foundation, implementacja referencyjna (używana w serwerze GlassFish).

Klasy encyjne

Klasy encyjne (entity classes):

- klasy odwzorowywane na table przechowywane w bazie danych:
- zwykłe klasy POJO (Plain Old Java Object),
- pola klasy nie mogą być publiczne,
- każdy obiekt encyjny ma jednoznacznie identyfikujący go klucz główny:
  - złożone klucze główne są reprezentowane przez oddzielne klasy, które muszą implementować metody `hashCode()` i `equals()`;
- oznaczone za pomocą adnotacji.

Istotne adnotacje JPA - klasa

Adnotacje na poziomie **klasy**:

- `@Entity` – oznaczenie klasy jako encyjnej (wymagana),
- `@Table` – własności tabeli w bazie danych, np.:
  - `name` – nazwa tabeli,
  - `indexes` – dodatkowe indeksy (domyślny indeks dla klucza głównego).

Istotne adnotacje JPA - pola

Adnotacje na poziomie **pól**:

- `@Id` – oznaczenie klucza głównego,
- `@GeneratedValue` – automatyczne generowanie wartości klucza głównego,
- `@Column` – własności kolumny w bazie danych, np.:
  - `name` – nazwa kolumny,
  - `nullable` – czy pole jest wymagane,
  - `unique` – czy kolumna przechowuje unikalne wartości,
  - `updatable` – czy wartość w kolumnie można modyfikować po zapisaniu nowego wiersza w tabeli;
- `@Temporal` – wymagane dla typów `Date` i `Calendar`:
  - umożliwia wykorzystanie bazodanowych typów do przechowywania daty/czasu, jeśli baza je oferuje;
- `@Transient` – oznaczenie pól klasy, które mają zostać pominięte przy mapowaniu obiektowo-relacyjnym.

Przykład klasy encyjnej

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Book {

    @Id
    private UUID id;

    private String title;

    @Column(unique = true)
    private String isbn;

}
```

Adnotacje `@NoArgsConstructor`, `@EqualsAndHashCode`, `@ToString`, `@Getter`, `@Setter` pochodzą z biblioteki *Project Lombok* i służą do generowania metod na etapie kompilacji.

Generowanie wartości klucza głównego

Atrybut `strategy` adnotacji `@GeneratedValue` określa sposób generowania wartości kluczy głównych:

- `GenerationType.IDENTITY` - bazuje na automatycznie zwiększanej (auto increment) wartości w kolumnie w bazie danych,
- `GenerationType.SEQUENCE` - bazuje na bazodanowej sekwencji do generowania kolejnych wartości,
- `GenerationType.TABLE` - wykorzystanie aktualizowanej wartości w odrębnej tabli w bazie danych,
- `GenerationType.AUTO` - implementacja JPA wybiera strategię w zależności od bazy danych.

Związki między encjami

Związki pomiędzy tabelami w bazie danych można odzwierciedlić jako **powiązania** między encjami:

- jednokierunkowe – jedna z klas encyjnych zawiera odwołanie do drugiej,
- dwukierunkowe – obie klasy encyjne posiadają wzajemne odwołania.

Adnotacje definiujące **powiązania**:

- `@OneToOne`,
- `@OneToMany`,
- `@ManyToOne`
- `@ManyToMany`.

Wybrane własności adnotacji:

- `mappedBy` - oznaczenie pola będącego właścicielem związku,
- `cascade` - operacje kaskadowe na elementach zależnych od siebie.

Przykład związku jednokierunkowego

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Employee {

    @Id
    private UUID id;

    private String name;

    private String surname;

    @MangToOne
    private Department department;

}
```

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Department {

    @Id
    private UUID id;

    private String name;

}
```

Związki dwukierunkowe

Związki dwukierunkowe:

- w relacyjnej bazie danych nie występuje koncepcja związku dwukierunkowego:
  - związki modelowane przy użyciu kluczy obcych;
- w obiektowym języku programowania dwukierunkowe związki są częstym rozwiązaniem,
- obecność atrybutu `mappedBy` wskazuje, że pole reprezentuje drugą stronę związku modelowanego przez inne pole,
- strona bez `mappedBy` jest stroną właścicielską – kontroluje związek między encjami.

Przykład związku dwukierunkowego

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Employee {

    @Id
    private UUID id;

    private String name;

    private String surname;

    @ManyToOne
    private Department department;

}
```

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Department {

    @Id
    private UUID id;

    private String name;

    @Getter
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;

}
```

Przykład związku cyklicznego

```java
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Employee {

    @Id
    private UUID id;

    private String name;

    private String surname;

    @ManyToOne
    private Employee superior;

    @OneToMany(mappedBy = "superior")
    private List<Employee> subordinates;

}
```

Jednostka trwałości (ang. Persistence Unit)

Jednostka trwałości (Persistence Unit):

- grupuje klasy encyjne odwzorowane na tabele w tej samej bazie danych,
- aplikacja może definiować kilka jednostek trwałości (używać kilku baz danych),
- określa sposób połączenia z bazą danych:
  - nazwa hosta, port, nazwa bazy, login, hasło, sterownik JDBC do połączenia,
  - źródło danych (ang. DataSource) dostarczane przez środowisko wykonawcze (np. serwer aplikacji);
- standardowo konfigurowana w pliku `persistence.xml` (w katalogu `META-INF` projektu),
- frameworki mogą udostępniać alternatywne sposoby konfiguracji, definiować ustawienia domyślne
  - np. Spring Framework: `application.properties` + domyślna jednostka trwałości obejmująca wszystkie klasy encyjne projektu.

Kontekst i tożsamość bazodanowa

Kontekst (ang. Persistence Context):

- zbiór zarządzanych (ang. managed) obiektów encyjnych,
- zmiany w zarządzanych obiektach encyjnych są śledzone i zapisywane do bazy danych po zatwierdzeniu transakcji.

Tożsamość bazodanowa (ang. Persistence Identity):

- tożsamość obiektu encyjnego wyrażona identyfikatorem, który jest powiązany z istniejącym kluczem głównym w bazie danych.

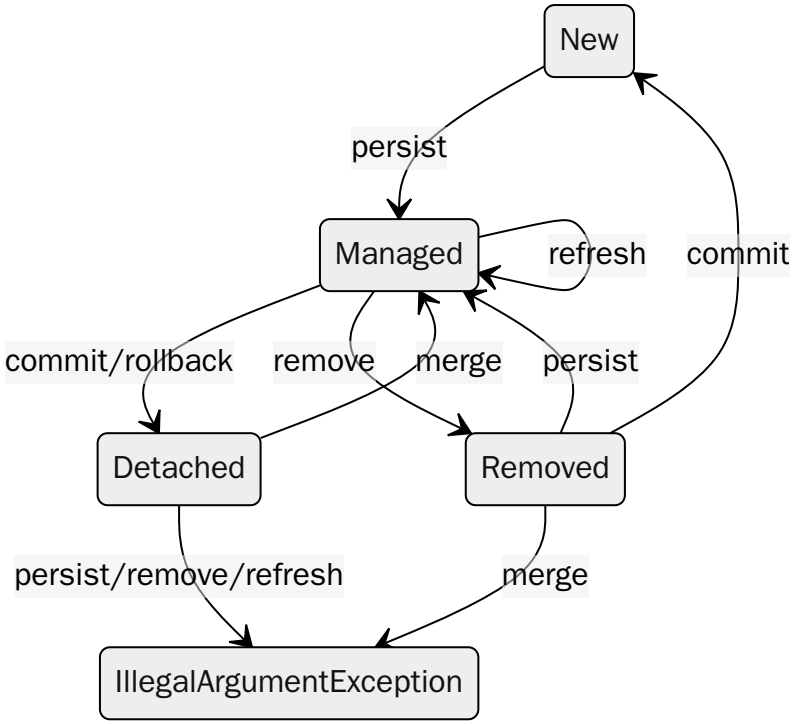
Stany obiektu encyjnego

stan encji	związana z Persistence Context	Posiada Persistent Identity
new	✗	✗
managed	✔	✔
detached	✗	✔
removed	✔	✔

Stan *removed* - przeznaczony do usunięcia przy zatwierdzeniu transakcji (obiekt istnieje w bazie danych, dopóki transakcja nie zakończy się sukcesem).

POLITECHNIKA GDAŃSKA

Cykl życia obiektu encyjnego



Konfiguracja - Jakarta EE

Konfiguracja dla aplikacji Jakarta EE uruchamianych w ramach serwera aplikacji:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
                                 https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
             version="3.0">
    <persistence-unit name="libraryPu" transaction-type="JTA">
        <jta-data-source>java:comp/env/jdbc/Library</jta-data-source>
        <properties>
          <property name="jakarta.persistence.schema-generation.database.action"
                    value="create" />
        </properties>
    </persistence-unit>
</persistence>
```

Konfiguracja - Java SE

Konfiguracja dla aplikacji Java SE:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
                                 https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
             version="3.0">
    <persistence-unit name="libraryPu" transaction-type="RESOURCE_LOCAL">
        <properties>
            <property name="jakarta.persistence.jdbc.driver"
                      value="org.postgresql.Driver" />
            <property name="jakarta.persistence.jdbc.url"
                      value="jdbc:postgresql://localhost:5432/books" />
            <property name="jakarta.persistence.jdbc.user"
                      value="admin" />
            <property name="jakarta.persistence.jdbc.password"
                      value="adminadmin" />
        </properties>
    </persistence-unit>
</persistence>
```

```xml
<properties>
    <property name="javax.persistence.jdbc.driver"
              value="com.mysql.cj.jdbc.Driver" />
    <property name="javax.persistence.jdbc.url"
              value="jdbc:mysql://localhost:3306/books?serverTimezone=UTC" />
</properties>
```

Konfiguracja - dialekty

Wykorzystanie implementacji Hibernate może wymagać ustawienia odpowiednich dialektów SQL.

MySQL:

```
<property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect"/>
```

PostgreSQL:

```
<property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQL82Dialect"/>
```

Entity Manager

Entity Manager - udostępnia podstawowe operacje zarządzania encjami:

- wywoływane operacje zmieniają stan obiektu encyjnego (new/managed/detached/removed),
- `void persist(Object o)` – zapis do bazy danych (new ⇨ managed),
- `<T> T merge(T entity)` – detached ⇨ managed
- `void remove(Object o)` – usunięcie encji (managed ⇨ removed)
- `void refresh(Object o)` – aktualizuje stan obiektu encyjnego na podstawie bazy,
- `<T> T find(Class<T> entityClass, Object key)` – wyszukiwanie na podstawie klucza głównego,
- `EntityTransaction getTransaction()` – zwraca obiekt transakcji:
  - większość frameworków umożliwia automatyczne zarządzanie transakcjami.

Entity Transaction:

- `begin()` – rozpoczyna transakcję,
- `commit()` – kończy transakcję, zapisuje zmiany do bazy, odłącza zarządzane obiekty encyjne,
- `rollback()` - wycofuje transakcję, odłącza zarządzane obiekty encyjne.

Pozyskanie instancji klasy EntityManager

Sposoby **pozyskania** obiektu `EntityManager`:

- Java SE:
  - poprzez fabrykę,
  - fabryka może być wykorzystywana przez wiele wątków,
  - instancja `EntityManager` może być bezpiecznie używana tylko przez jeden wątek;
- Java EE:
  - środowisko wykonawcze (np. serwer aplikacji) dostarcza instancje klasy `EntityManager`.

```
EntityManagerFactory factory=Persistence.createEntityManagerFactory("booksPu");
EntityManager em=factory.createEntityManager();
```

```
@RequestScoped
public class BookService {

    @PersistenceContext(name = "bookPu")
    private EntityManager em;

}
```

Operacje na bazie

Przykładowa klasa:

```
@NoArgsConstructor
@EqualsAndHashCode
@ToString
@Getter
@Setter
@Entity
public class Book {

    private String title;

    @Id
    private String isbn;

}
```

Operacje na bazie w Java SE

Uzyskanie obiektu fabryki:

```
EntityManagerFactory emf=Persistence.createEntityManagerFactory("testPu");
EntityManager em;
```

Zapisanie nowego obiektu w transakcji:

```
em = emf.createEntityManager();
em.getTransaction().begin();
em.persist(new Book("Ruda Sfora", "1"));
em.getTransaction().commit();
em.close();
```

Wyszukanie obiektu w bazie:

```
em = emf.createEntityManager();
Book wolf = em.find(Book.class, "1");
System.out.println(wolf);
em.close();
```

Zamknięcie fabryki na koniec programu:

```
emf.close();
```

JPQL - Java Persistence Query Language

Java Persistence Query Language (JPQL):

- obiektowo zorientowany język zapytań o składni nawiązującej do języka SQL,
- opiera się na zdefiniowanym w projekcie modelu klas encyjnych,
- umożliwia odpytywanie bazy danych (`SELECT`), aktualizację (`UPDATE`) i usuwanie (`DELETE`) encji,
- zapytania automatycznie tłumaczone na język SQL:
  - z uwzględnieniem automatycznych złączeń tabel w czasie trawersowania związków między encjami.

JPQL - klasa encyjna

```
@Entity
public class Product {

  @Id
  UUID id;

  String name;

  String description;

  Integer price;

  Integer amount;

}
```

JPQL - podstawowe zapytania

Pobranie wszystkich produktów:

```
SELECT p FROM Product p
```

Pobranie produktów o określonej nazwie:

```
SELECT p FROM Product p WHERE p.name = :name
```

JPQL - podstawowe zapytania cd

Pobranie produktów o nazwie zgodnej z wyrażeniem regularnym – operator `LIKE`:

```
SELECT p FROM Product p WHERE p.name LIKE :name
```

Wyszukanie produktów, których opis zawiera frazę *VT-x* w dowolnym miejscu:

```
SELECT p FROM Product p WHERE p.description LIKE '% VT-x %'
```

Bez uwzględniania wielkości znaków:

```
SELECT p FROM Product p WHERE LOWER(p.name) LIKE LOWER('%Laptop%')
```

Pobranie produktów, których stan jest niski (poniżej 5 sztuk), posortowanych według nazw:

```
SELECT p FROM Product p WHERE p.amount < 5 ORDER BY p.name
```

Wywołanie zapytań

Wywołanie zapytań bez parametrów:

```
String queryString = "SELECT p FROM Product p";
Query query = em.createQuery(queryString, Product.class);
List<Product> products = query.getResultList();
```

Wywołanie zapytań z parametrami:

```
String queryString = "SELECT p FROM Product p WHERE p.name LIKE :name";
Query query = em.createQuery(queryString, Product.class);
query.setParameter("name", "Laptop");
List<Product> products = query.getResultList();
```

Zliczanie:

```
String queryString = "SELECT COUNT(p) FROM Product p WHERE p.name LIKE :name";
Query query = em.createQuery(queryString, Long.class);
query.setParameter("name", "Laptop");
Long count = query.getSingleResult();;
```

Wywołanie zapytań cd

OFFSET oraz LIMIT:

```
String queryString = "SELECT p FROM Product p";
Query query = em.createQuery(queryString, Product.class);
query.setFirstResult(21);
query.setMaxResults(10);
List<Product> products = query.getResultList();
```

Fluent syntax:

```
em.createQuery("SELECT p FROM Product p WHERE p.name LIKE :name",Product.class)
        .setFirstResult(21)
        .setMaxResults(10)
        .setParameter("name","Laptop")
        .getResultList();
```

Dla zapytań UPDATE oraz DELETE:

```
String queryString = "DELETE FROM Product p";
Query query = em.createQuery(queryString);
int changedRows = query.executeUpdate();
```

Zapytania nazwane

Zapytania używane w wielu miejscach aplikacji można zdefiniować przy użyciu adnotacji `@NamedQuery`:

```
@NamedQueries({
        @NamedQuery(
                name = "Product.findAll",
                query = "SELECT p FROM Product p"
        ),
        @NamedQuery(
                name = "Product.findByName",
                query = "SELECT p FROM Product p WHERE p.name = :name"
        )
})
@Entity
public class Product {
    //...
}
```

```
Query query=em.createNamedQuery("Product.findAll");
```

Bazy danych: standalone vs embedded

Wybór bazy danych:

- w środowisku produkcyjnym typowo wykorzystuje się samodzielne bazy danych (standalone DBMS)
  - np. Oracle Database, PostgreSQL, MySQL, MS SQL;
- w czasie wytwarzania aplikacji/testów często wygodniejsze jest użycie bazy danych osadzonej w aplikacji (embedded)
  - np.: Java DB, H2, HSQLDB (HyperSQL).

Baza danych w trybie *embedded*:

- nie wymaga osobnej instalacji, konfiguracji, uruchomienia,
- baza danych startuje automatycznie razem z aplikacją,
- dane przechowywane na dysku lub w pamięci RAM,
- pamięć RAM::
  - szybszy dostęp,
  - zawartość bazy czyszczona w chwili zamknięcia aplikacji,
  - przydatne do testów integracyjnych.

JPA w aplikacji Java EE

JPA w aplikacji Java EE:

- połączeniem z bazą danych zarządza serwer aplikacji:
  - konfiguracja danych dostępowych zależna od wybranego serwera,
  - konfiguracja nie występuje w kodzie samej aplikacji:
  - przenośność pomiędzy serwerami (+),
  - rozproszenie konfiguracji (-);
  - serwer wykorzystuje pulę połączeń z bazą danych do obsługi wielu użytkowników,
  - połączenie jest udostępniane aplikacji jako zasób JNDI (Java Naming and Directory Interface).

JPA w aplikacji Java EE cd

JPA w aplikacji Java EE:

- wstrzykiwanie obiektu klasy EntityManager:

```
@PersistenceContext
EntityManager em;
```

- transakcje na serwerze Java EE:
  - zarządzane przez kontener – w warstwie EJB (automatycznie),
  - zarządzane przez użytkownika (dewelopera),
  - zarządzane w beanach CDI (`@Transactional`);
- serwery Java EE umożliwiają realizację transakcji rozproszonych:
  - operacje na wielu bazach danych w jednej transakcji,
  - jedna transakcja obejmująca wiele serwerów aplikacji, z których każdy
  - wykorzystuje własną bazę/bazy danych.

Transakcje zarządzane przez użytkownika

Wstrzyknięcie obiektu transakcji:

```
@Resource
UserTransaction utx;
```

Operacje na bazie danych:

```
try {
    utx.begin();
    //...
    utx.commit();
} catch (Exception ex) {
    utx.rollback();
}
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/persistence/3.1/

Dokumentacja:

- https://hibernate.org/orm/documentation/5.6/.
- https://hibernate.org/orm/documentation/6.3//.

Uzupełnienie informacji:

- https://eclipse-ee4j.github.io/jakartaee-tutorial/#persistence
- https://www.baeldung.com/jpa-entities
- https://www.baeldung.com/jpa-composite-primary-keys
- https://www.baeldung.com/jpa-strategies-when-set-primary-key
- https://www.baeldung.com/jpa-one-to-one
- https://www.baeldung.com/jpa-join-column
- https://www.baeldung.com/jpa-joincolumn-vs-mappedby
- https://www.baeldung.com/jpa-queries